Enhancing 7-DOF Robotic Arm Inverse Kinematics: A Machine Learning and Optimization Approach

Kaan Ural
School of CSE, University of Salford, Manchester M5 4WT, UK
k.ural@edu.salford.ac.uk

Inverse kinematics is the computational process in robotics used to determine the joint configurations necessary for a robotic arm to achieve a desired position and orientation. This study addresses the challenges associated with solving inverse kinematics for a 7 degrees of freedom (DOF) robotic system. To overcome these challenges, machine learning techniques and optimization algorithms are employed. Specifically, Feedforward Neural Networks are utilized for their predictive capabilities, while Simulated Annealing and Particle Swarm Optimization algorithms are implemented to efficiently search the complex solution space. The dataset, crucial for training and evaluating these models, is generated using forward kinematic equations, then verified by using simulations from Gazebo, a robust robot simulation environment. This combination provides a rich, varied dataset that mimics real-world operations. The deployment of these advanced computational techniques demonstrates potential improvements in accuracy and efficiency over traditional methods, thus offering promising directions for future research in robotic motion planning.

Index Terms – robotic arm; inverse kinematic; machine learning; neural networks; optimization; ROS; Gazebo;

I. INTRODUCTION

Inverse kinematics (IK) is a fundamental computational technique in robotics, enabling the precise control of robotic arms by determining the joint parameters that provide a desired position and orientation of the robot's end effector. The accuracy of IK is crucial in complex robotic systems, especially those with seven degrees of freedom (7-DOF). These systems are increasingly common in advanced applications such as automated manufacturing and medical robotics. However, the complexity of 7-DOF systems also presents significant challenges, including a higher potential for inaccuracies, higher computational demands and the increased likelihood of encountering multiple or no viable solutions which represent a major issue in ensuring their effectiveness.

Traditional algorithms often struggle to manage these complexities, leading to a growing interest in alternative and innovative approaches. As in [1], recent advancements in the application of Artificial Intelligence (AI) and Machine Learning (ML) techniques have shown significant promise in enhancing the precision and efficiency of IK solutions for these high-DOF systems. This study explores various AI techniques and optimization algorithms, including Feedforward and Recurrent Neural Networks, Random Forest Regression, and evolutionary algorithms like Genetic Algorithm and Particle Swarm Optimization, algorithms as solutions to these challenges and assesses their efficacy in solving the IK challenges of 7-DOF robotic arms.

The significance of this investigation lies in its potential to contribute to the field of robotic motion planning, enhancing the accuracy and efficiency of inverse kinematics (IK) solutions. This provides valuable insights that could lead to more precise and complex movements in robotic systems, influencing future design and control strategies.

A. Literature Review

The literature reveals diverse approaches to addressing machine learning techniques applications on IK problems, with varying degrees of success.

For example, globally optimal solutions using advanced polynomial optimization have demonstrated high efficacy in test poses for 7DOF serial manipulators [2]. Other studies [3], presents a new inverse kinematics optimization method for redundant robots using Gaussian Damped Least Squares with an enhanced step-size coefficient. This method improves speed and accuracy, achieving fast convergence with high precision on a 7R redundant robot, outperforming common IK methods. Further research yields that the PSO algorithm leverages intelligent cooperation among particles to enhance efficiency in maneuvering poses and performing tasks and simulation results demonstrate the effectiveness of this method[4]. In [5], an artificial neural network (ANN) with a backpropagation algorithm is used to solve the inverse kinematics of a 6R robot manipulator with offset wrist, which lacks a closed-form solution. Different robot end-effector orientations were compared, resulting in satisfactory simulation results with low error rates[5]. This study [6] improves robot inverse kinematics by using a genetic algorithm to optimize variable damping coefficients for each singular vector, resulting in reduced tracking errors and smoother velocity transitions in simulations.

The rest of the paper is organised as follows Section II outlines the methodology, providing definitions of key kinematic concepts, details on model development, and descriptions of data handling procedures. Section III presents the results, showcasing the performance of the deployed techniques. Section IV discusses the implications of these results and explores potential avenues for future research and offers concluding remarks, summarizing the findings and their potential impact on the field of robotics.



Figure. 1.1 The KUKA iiwa 7 R800 Robot. Photo courtesy of KUKA.

II. METHODS

A. *Kinematic Definitions and Preliminaries*

Establishing coordinate frames for a robotic arm is a fundamental step in robotic kinematics, which involves defining a set of reference frames to describe the position and orientation of the robot's joints and links. This process is essential for controlling the robot and for understanding its motion in three-dimensional space.

The Denavit-Hartenberg (DH) convention[7] is a systematic method to assign coordinate frames to a robotic manipulator's links. This method simplifies the mathematical modeling of the robot's geometry and is widely used for the kinematic analysis of robotic arms. The DH convention utilizes four parameters for each joint of a robotic arm:

θ (theta) - the joint angle, which is the angle between the two adjacent link's common normal, i.e., the perpendicular line to the intersection of the two link axes.

d (distance) - the link offset, which is the distance from one link frame to the next along the previous link's axis.

a (alpha) - the link length, which is the distance from one link frame to the next along the common normal.

α (alpha) - the link twist, which is the angle between the two link axes.

These parameters are used to create a link transformation table that describes the relationship between adjacent link coordinate frames. The transformation from one link frame to the next is represented by a homogeneous transformation matrix, which is a 4x4 matrix that combines rotation and translation. The matrix is constructed using the DH parameters and consists of the following:

- A rotation about the z-axis by the angle θ.
- A translation along the z-axis by the distance d.
- A translation along the x-axis by the distance a.
- A rotation about the x-axis by the angle α.

The product of these individual transformations for each link, when multiplied together, gives the overall transformation matrix that describes the position and orientation of the end-effector relative to the base frame of the robot.

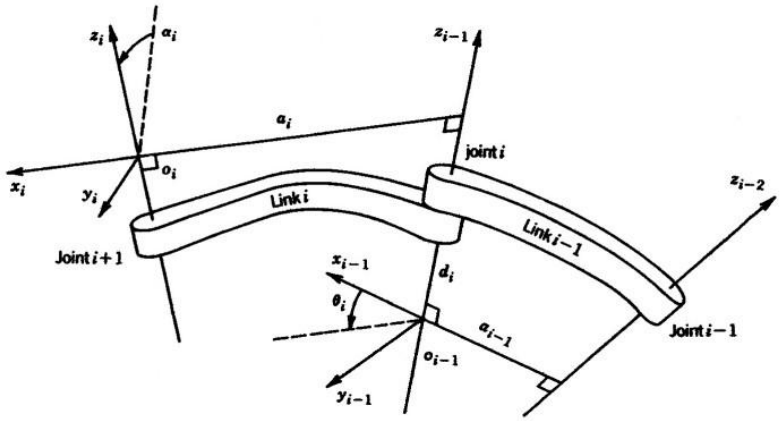


Figure 2.1. Denavit-Hartenberg frame assignment[8].

As shown in Figure 2.1, the DH convention assigns four parameters to each joint, which are then used to define the transformation from one joint to the next. These parameters are crucial in the calculation of the robot's kinematics and allow precise control over its movement and orientation.

In DH convention, each homogeneous transformation is represented as a product of four basic transformations

$$
\begin{aligned}
A_i &= Rot_{z,\theta_i}\,\mathrm{Trans}_{z,d_i}\,\mathrm{Trans}_{x,a_i}\,Rot_{x,\alpha_i} \qquad (1)\\
&= \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} & 0 & 0 \\ s_{\theta_i} & c_{\theta_i} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&\quad \times \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c_{\alpha_i} & -s_{\alpha_i} & 0 \\ 0 & s_{\alpha_i} & c_{\alpha_i} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} c_{\theta_i} & -s_{\theta_i} c_{\alpha_i} & s_{\theta_i} s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i} c_{\alpha_i} & -c_{\theta_i} s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}
\end{aligned}
$$

The transformation matrix A_i, detailed in equation (1)[8], contains these parameters into a 4x4 matrix combining rotation and translation movements during transition from one joint to another. This matrix enables the calculation of the position and orientation of each link in the robotic arm relative to a common base frame. Table 2.1 lists the DH parameters for the Kuka LBR iwa 7 R800 robot, which will be employed in the

subsequent dataset generation and model validation steps of this research.

Ensuring the correctness of these parameters is critical, as any inaccuracies can lead to significant errors in the IK solutions. Therefore, the parameters were validated against the robot's physical movements and simulation data from Gazebo to guarantee their accuracy. An example of this process is illustrated for Joint 1, where θ_1 represents the rotation around the z-axis by angle q_1, with a link offset d_1 of 0.34 meters.

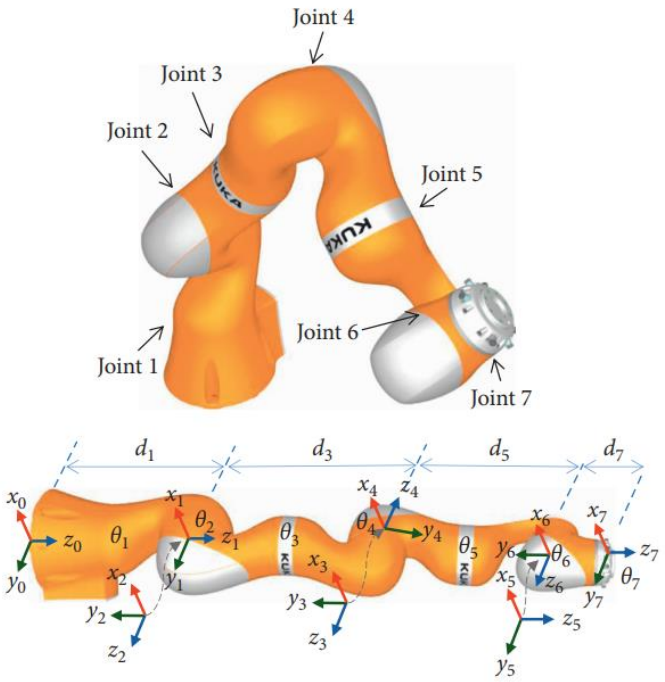


Figure 2.2. : Establishing coordinate frames for the Kuka LBR iiwa 7 R800[9].

TABLE I
DH PARAMETERS FOR EACH JOINT OF THE KUKA LBR IWA 7 R800 ROBOT [9][10].

Link i	θ_i	d_i	a_i	α_i	Joint Limits
1	q_1	0.34	0	-90	±170 °
2	q_2	0	0	90	±120 °
3	q_3	0.4	0	90	±170 °
4	q_4	0	0	-90	±120 °
5	q_5	0.4	0	-90	±170 °
6	q_6	0	0	90	±120 °
7	q_7	0.126	0	0	±175 °

B. Dataset Generation

Before exploring the details of how the dataset was generated, it is important to first introduce the simulation tools used in this study. The Robot Operating System (ROS) is an open-source middleware suite offering services tailored for diverse computing environments, including hardware abstraction, low-level device control, commonly used functionality, message-passing between processes, and package management[11]. It facilitates the structured development of robotic software by providing a wide array of tools and libraries designed to ease the creation of sophisticated and durable robot behavior on various robotic platforms.

Complementing ROS Noetic, Gazebo offered the high-fidelity simulation environment required for this research. Gazebo is an advanced robotics simulation software that has become a standard for robot simulation due to its robust physics engine, high-quality graphics, and its ability to simulate populations of robots in complex environments[12].

Within this framework, the KUKA iiwa 7 R800 robot's model, which was created by researcher at Technical University of Munich[13], was integrated into Gazebo. The ROS-based MoveIt package, known for its advanced motion planning capabilities, was employed to direct the robot arm's operations. This setup facilitated the execution of a spectrum of movements that are both sequential and random, thus creating variety in the dataset.

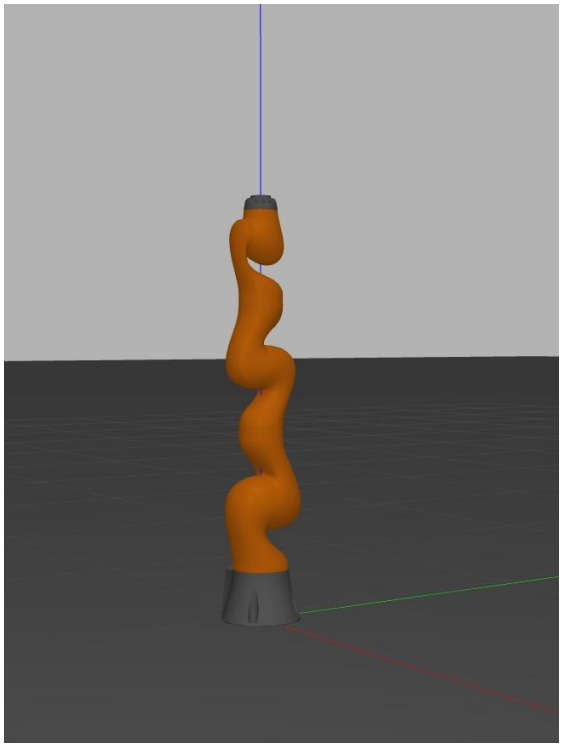


Figure 2.3. : KUKA LBR iiwa 7 R800 in Gazebo Environment[14].

The data capturing phase involved collecting information on the robot's joint states and the end-effector's pose through the ROS messaging system. The rosbag tool was utilized for this purpose, recording the data published over ROS topics, which effectively captured the states of the robot's dynamics during various movements.

The obtained dataset encompasses a wide array of the robot arm's maneuvers, ensuring a rich repository of scenarios for training machine learning models. The successive subsection will expand on the preprocessing steps taken to refine this dataset for optimal use in machine learning contexts.

The successive subsection will expand on the preprocessing steps taken to refine this dataset for optimal use in machine learning contexts.

Preprocessing steps were applied to prepare the dataset for the next stage of model training and evaluation, details of which are expounded in the following subsection.

The position data for the end effector was obtained from the Gazebo simulation environment, while the joint angles of the robot arm were collected from the iiwa_controller, a ROS control interface specifically designed for the KUKA iiwa 7 R800. In Gazebo, link data is broadcasted in a single message that contains data of every link in the environment, making it necessary to the develop specialized Python scripts to isolate and extract the position of the arm's seventh link, which serves as the end effector. The ability of rosbag tool to save data along with timestamps, was instrumental in synchronizing the joint angle data, published at a rate of 100 Hz, with the link position data, published at a higher rate of 1000 Hz. This synchronization was achieved with the aid of the Pandas library, ensuring accurate correlation between the two datasets for subsequent analysis.

Now that we the data ready to be preprocessed to make it suitable for ML algorithms, it is appropriate to proceed to data preprocessing.

C. Data Preprocessing

After collecting the dataset, the next crucial step is to preprocess the data to ensure it is suitable for machine learning analysis. This subsection covers the following key points:

1) *Feature Selection:* Feature engineering is a critical step for effective machine learning training. In this study, the feature selection was straightforward and driven by the intrinsic properties of the robotic system being modeled. The selected features included the seven joint angles of the KUKA iiwa 7 R800 robot, and the three-dimensional coordinates (X, Y, and Z) of the robot's end effector link.

2) *Normalization/Standardization:* To ensure that all features contributed equally to the model's learning process, normalization and standardization techniques were applied using tools from the scikit-learn library. Specifically, StandardScaler was employed to normalize features by removing the mean and scaling to unit variance. This standardization technique is particularly useful for algorithms that assume data is normally distributed and operates on the principle that features need to be on the same scale for optimal performance.

In certain models where we needed to ensure that the feature values remained within a specific range (typically 0 to 1), MinMaxScaler was utilized. This scaling compresses all initial input feature ranges into the bound range, maintaining the distribution but aligning the scale of each feature. This was essential for models that are sensitive to variable sizes, like neural networks, to ensure that the gradients calculated during training didn't unevenly impact the learning process.

3) *Splitting the Dataset:* An essential part of preparing the dataset for machine learning is dividing it into training, validation, and test sets. This process is critical for evaluating the performance of the models in a structured way, allowing us to fine-tune parameters and test the models' ability to generalize to new data.

a) *Training Set:* 70% of the dataset was allocated to the training set. This portion of the data is used to train the machine learning models, allowing the algorithms to learn the relationships between the features (joint angles and XYZ coordinates of the end effector) and the target outputs. The large proportion of data dedicated to training ensures that the models have ample examples to learn from, which is particularly important given the complexity of the movements being modeled.

b) *Validation Set:* 15% of the dataset was reserved for the validation set. This subset is used during the model training process but is not directly involved in learning. Instead, it serves to validate the model's performance after each epoch of training, allowing for the adjustment of hyperparameters and helping prevent overfitting. The validation set acts as a check during training to ensure that improvements in the model's performance are genuine and not just a result of memorizing the training data.

c) *Test Set:* The remaining 15% of the dataset was used as the test set. This set is crucial as it is only used after the model training is complete. It provides an unbiased evaluation of a final model's performance and is critical for assessing how well the model is likely to perform in real-world scenarios, where it encounters data it has never seen before.

D. Model Development

This study employs a variety of various machine learning models and optimization algorithms, each selected for their proven effectiveness in handling complex, nonlinear problems such as 7-DOF robotic arm inverse kinematics. Each method

has been tailored to take full advantage of the preprocessed features—joint angles and end effector positions.

Below, each model's choice is justified, and their development processes are described, including architectural details, and computational considerations.

1) *Neural Networks:*

a) Feedforward Neural Network (FNN):

Utilized for their superior capability to approximate complex nonlinear functions[16]. This makes them well-suited for modeling the nonlinear relationships between the joint configurations of a 7-DOF robotic arm and the positional outcomes of its end effector, crucial for predicting precise movements accurately.

The FNN designed for this study consists of five layers with specific neuron configurations: the network begins with three successive layers each containing 300 neurons, followed by a fourth layer with 128 neurons, and a fifth layer with 32 neurons. Each of these layers utilizes the ReLU (Rectified Linear Unit) activation function, chosen for its effectiveness in introducing non-linearity while also helping to mitigate the vanishing gradient problem during training[17]. With purpose of comparing activation function performances, a secondary FNN model was also trained with Tanh activation function. Tanh is defined in Eq. 3 and it outputs values between -1 and 1, providing a normalized output that helps in maintaining stable gradients.. The output layer, which matches the dimensions of the end effector's positional data, uses a linear activation function to directly predict the positions.

The FNN processes input data through successive layers, where each layer's output becomes the subsequent layer's input. The operation within each neuron involves a weighted sum of the inputs plus a bias term, followed by the activation function. The ReLU activation function, defined as f(x)=max(0,x), is used in the hidden layers to introduce non-linearity, allowing the network to learn more complex patterns. The network's final output is computed using a linear combination in the output layer, directly correlating to the end effector's positional coordinates.

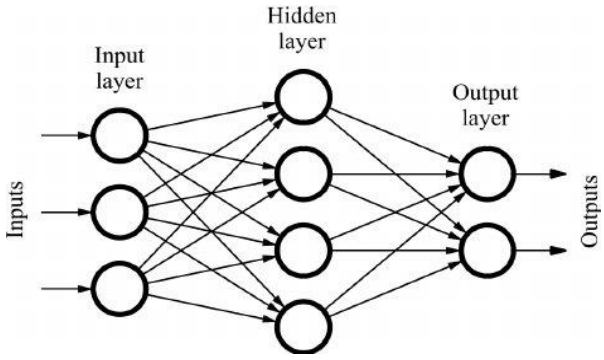


Figure 2.4. : Feedforward neural network model[15].

ReLU activation function is given in Eq . 2:

$$f(x) = max(0, x) \qquad (2)$$

Tanh activation function is given in Eq . 3:

$$f(x) = (e^{x} - e^{-x}) \,/\, (e^{x} + e^{-x}) \qquad (3)$$

Mathematically, the operation of a neuron, i.e. output layer transformation in layer l can be described in Eq . 4 as follows:

$$a[l] = g^{[l]}\left(W^{[l]}a^{[l-1]} + b^{[l]}\right) \qquad (4)$$

Where $a[l]$ is the activation of layer l, $W^{[l]}$ and $b^{[l]}$ are the weights and biases of layer $g^{[l]}$ is the activation function[18].

The model was implemented using TensorFlow's Keras API, providing a streamlined approach to building and training neural networks. The model is compiled with SGD (Stochastic Gradient Descent) as the optimizer, featuring a learning rate of 0.017 and a momentum of 0.9 to enhance the convergence speed. The 'mean squared error' is used as the loss function to minimize the average squared difference between estimated values and what is observed.

b) Recurrent Neural Network (RNN) and their advanced variant Long Short-Term Memory (LSTM): Given the sequential nature of robotic movements, RNNs, particularly LSTMs, are employed to model the temporal dependencies of joint movements, which are crucial for dynamic tasks where the current state often depends on previous states. Their ability to maintain information in 'memory' over time is crucial for accurately modeling the sequence of movements in a robotic arm.

The architecture consists of a Simple RNN layer followed by two dense layers. Specifically: The first layer is a Simple RNN with 128 units, employing the ReLU activation function to introduce non-linearity without hindering gradient flow. This layer processes the sequential data input, capturing the temporal relationships. Following the RNN layer, there is a dense layer with 128 neurons, also using the ReLU activation function, which helps in further abstracting the features learned by the RNN layer. A subsequent dense layer with 32 neurons uses a linear activation to prepare for the final output. The final output layer consists of 7 neurons (corresponding to the 7 dimensions needed for 7 angles, presumably for the robot's joint angles or positions), with a linear activation function to output the continuous values directly.

The Simple RNN operates by processing inputs through sequences, where each neuron in the RNN layer uses its previous output as part of the input for the current step, effectively maintaining a form of 'memory' about earlier inputs:

$$h_t = ReLU(W \cdot [h_t - 1, x_t] + b) \qquad (5)$$

where h_t is the output at time step t, W represents the weights, b is the bias, and x_t is the input at time t[19].

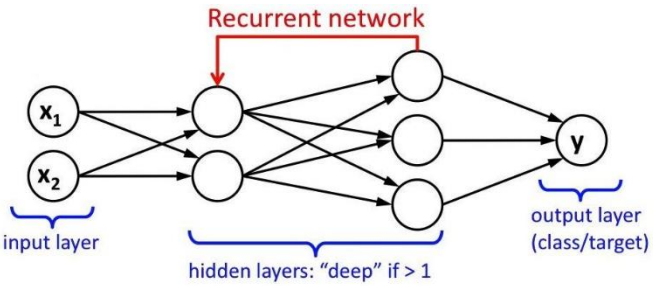


Figure 2.5. : Recurrent neural network model[20].

The model was implemented using TensorFlow's Keras API for ease of use and flexibility. The model is compiled with the Adam optimizer, a common choice for training neural networks due to its efficient handling of sparse gradients on noisy problems.
The loss function used is 'mean squared error' (MSE), which is standard for regression problems, and the metric for evaluating the model during training is 'mean absolute error' (MAE).

The training process involves running for 200 epochs with a validation split of 20%, providing feedback on model performance and helping prevent overfitting.

Long Short-Term Memory networks are an advanced type of RNN known for their ability to capture long-term dependencies in sequence data, addressing the vanishing gradient problem that affects simpler RNNs. This characteristic makes LSTMs particularly suitable for complex sequence modeling tasks, such as predicting the sequential movements of a robotic arm where current actions are dependent on a history of previous movements.

The LSTM model's architecture mirrors the structure of the previously described Simple RNN model but incorporates LSTM cells instead of simple RNN units.

The LSTM operates with a sophisticated internal mechanism featuring gates that regulate information flow: forget gate, input gate, and output gate. These gates decide what information is to be kept or discarded, how to update the internal memory cell, and what to output at each sequence step, enabling effective handling of both short-term and long-term dependencies[21].

Mathematical operations within an LSTM unit involve complex interactions between these gates, which are essential for maintaining relevant information across lengthy data sequences and mitigating gradient-related issues.

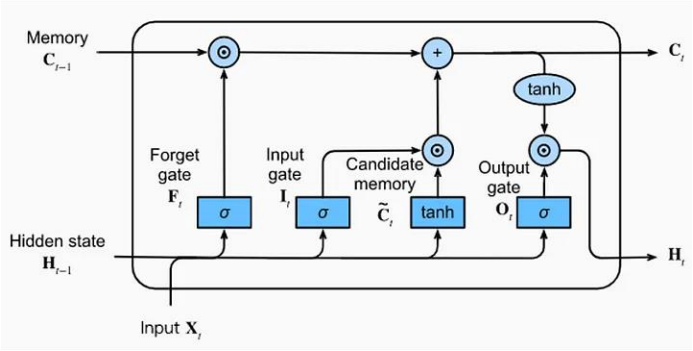


Figure 2.6. : Long Short Term Memory neural network model[22].

LSTM model in this study follows the same implementation framework as the RNN, using TensorFlow's Keras API for intuitive model construction.
Compiled with SGD optimizer, the model settings—learning rate of 0.01 and momentum of 0.9—are optimized for efficient learning dynamics.

The training regimen involves 60 epochs, balancing the need for thorough learning without overfitting, given the robust capabilities of LSTM in handling sequence predictions.

2) *Regression Models:*

a) Random Forest Regression:

Random Forest Regression is chosen for its robustness and effectiveness in dealing with both linear and non-linear data relationships. It operates by building multiple decision trees and outputting the average prediction of these trees, reducing overfitting and improving generalization. This ensemble method is particularly suited to the complexities involved in modeling robotic arm movements, where interactions between variables are highly nonlinear.

The Random Forest model consists of an ensemble of decision trees, each trained on different samples of the data:

Number of Trees: The model uses 20 trees (n_estimators=20), balancing complexity and computational efficiency.

Out-of-Bag Scoring: The oob_score=True parameter enables out-of-bag estimates for model accuracy, leveraging bootstrap sampling to evaluate the model's performance on unseen data during training.

Random State: A fixed random_state ensures reproducibility of your results by initiating a consistent random number generation.

Random Forest operates by creating multiple decision trees during training. Each tree predicts an output, and the final model output is the average of these predictions:

The out-of-bag (OOB) error is used as an additional metric to assess the model's performance without needing a separate validation set. It estimates accuracy by using data not included in the bootstrap sample for each tree.

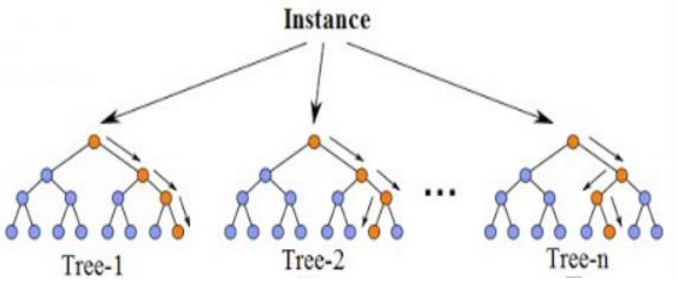


Figure 2.7. : Random Forest Regression Model[23].

The primary computational equation involves averaging the predictions from all trees:

$$\hat{y} = \frac{1}{N}\sum_{i=1}^{N} \hat{y}_i \qquad (6)$$

where $\hat{y}_i$ the prediction from the i-th tree and N is the total number of trees (20 in this case)[24].

The model is implemented using the RandomForestRegressor from the scikit-learn library, tailored to handle both numerical and transformed categorical data.

The regressor is fitted to the data, prepared as described, with the aim of minimizing prediction in the dataset.

b) Decision Tree Regressor: utilized for its clarity and effectiveness in modeling complex, nonlinear relationships with a clear hierarchical decision structure. This model is excellent for understanding the intricate dependencies between the joint configurations of a robotic arm and its movements, thanks to its interpretative nature, which allows detailed insight into how decisions are made at each node.

The Decision Tree involves a series of binary decisions that progressively split the data into more homogeneous subsets based on specific features:

Root and Nodes: The process starts at the root node, containing the entire dataset, which then splits into further nodes based on the selected features and thresholds.

Depth of Tree: Configured with a max_depth of 18, this setting helps the tree learn detailed data specifics without becoming overly complex, which could lead to overfitting.

Leaf Nodes: The min_samples_leaf parameter is set to 15, ensuring that each leaf node has enough samples to make a reliable prediction, further preventing overfitting by smoothing out the model's predictions.

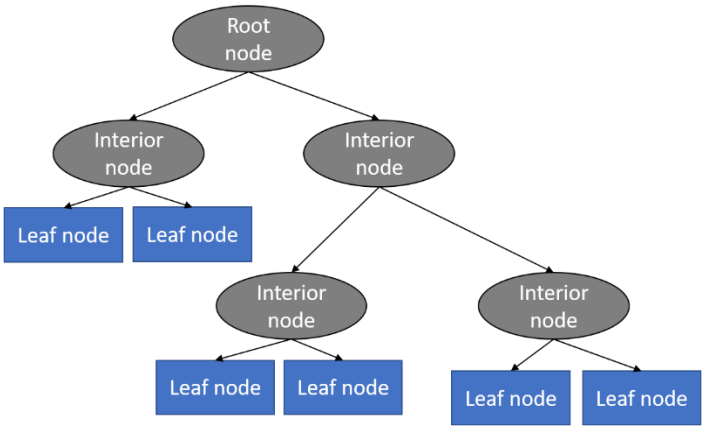


Figure 2.8. : Decision Tree Regression Model[25].

A Decision Tree Regressor constructs a tree by choosing the best splits based on a criterion that minimizes variance within each subset, aiming to create nodes with the most homogeneous outcomes possible:

The decision at each node is determined by selecting the feature and threshold that result in the lowest mean squared error (MSE), calculated as:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \qquad (7)$$

where Y_i are the actual values, $\hat{Y}_i$ are the predicted values from the node, and n is the number of samples at that node[26].

The Decision Tree Regressor is implemented using the DecisionTreeRegressor class from the scikit-learn library, configured with specific parameters to balance complexity and prediction accuracy. This setup is implemented to prevent the model from overfitting, allowing it to generalize better to unseen data. The use of a fixed random_state ensures that the results are reproducible. The final output includes the calculation of the mean squared error on the test set, providing a quantitative measure of the model's prediction accuracy.

Support Vector Regression (SVR): Selected for its proficiency in handling both linear and non-linear relationships by fitting the data within a specified margin of tolerance, known as epsilon. This method is particularly effective in high-dimensional spaces, making it ideal for the complex and often non-linear task of predicting robotic arm movements where precise accuracy is crucial.

c)The SVR in this study utilizes the Radial Basis Function (RBF) kernel, which allows the algorithm to address non-linear relationships by mapping inputs into a higher-dimensional space where linear regression can be applied. The architecture includes:

Kernel Function: The RBF kernel, a popular choice for non-linear data, transforms the input features into a higher-dimensional space.

Regularization Parameter (C): Set at 100, this parameter controls the trade-off between achieving a low error on the training data and minimizing the model complexity for better generalization.

Epsilon (ε): Set at 0.1, epsilon specifies the epsilon-tube within which no penalty is associated with predictions, allowing some errors to be tolerated within this margin.

Multi-Output Strategy:

To handle multiple regression outputs (such as predicting several joint angles or positions simultaneously), the SVR model is wrapped in a MultiOutputRegressor. This wrapper extends the capabilities of SVR, which is typically a single output regressor, to handle multi-dimensional regression tasks efficiently:

MultiOutputRegressor: This approach involves fitting one regressor per target. This means that for each output variable (joint angle or position), a separate SVR model is trained, but all using the same configuration and training set.

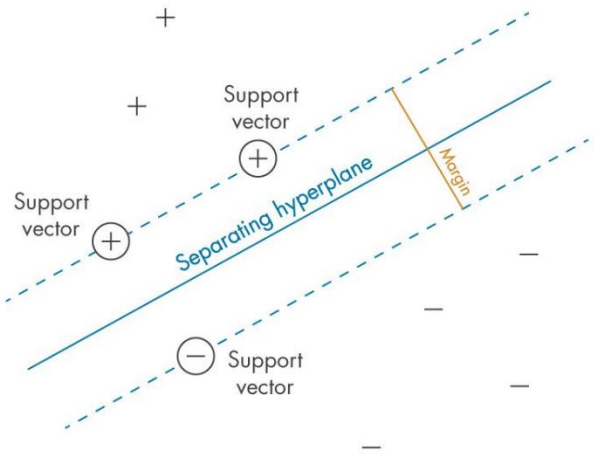


Figure 2.9. : Support Vector Machine Model[27].

The SVR operates by attempting to fit the regression function within an epsilon-margin while also trying to minimize the norm of the coefficient vector (model complexity). The optimization problem can be expressed as in Eq.8:

$$\min_{\mathbf{w},b,\xi,\xi^*} \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{n}(\xi_i + \xi_i^*) \quad (8)$$

Subject to constraints defined in Eq.9:

$$\mathbf{w}^T\phi(\mathbf{x}_i) + b - y_i \leq \epsilon + \xi_i^* \quad (9)$$

Where,

y_i are the actual target values,

$\mathbf{x}_i$ are the input data points,

$\phi(\mathbf{x}_i)$ is the feature transformation applied to xi , typically implicit in the kernel function used,

b is the bias term of the model,

ϵ is the margin of tolerance within which no penalty is applied to predictions,

Finally $\xi_i + \xi_i^*$ are again the slack variables, where accounts for predictions that are too high, and for predictions that are too low compared to the actual value[28].

Implemented using the SVR class from the scikit-learn library, wrapped in a MultiOutputRegressor to accommodate the multiple outputs of the robotic kinematics predictions.

3) Optimization Algorithms:

Before exploring the use of optimization algorithms in robotics, it's crucial to understand forward kinematics. This mathematical framework calculates the position and orientation of a robot's end-effector based on its joint configurations. This knowledge is essential because it allows us to predict the mechanical outcomes of joint movements, setting the stage for applying optimization algorithms that aim to enhance robotic precision and efficiency in various applications.

a) Genetic Algorithm (GA): Genetic Algorithms are inspired by natural selection and are widely used for optimization problems with complex search spaces, such as robotic kinematics. They are particularly effective for finding global optima in problems where the solution space is non-linear and multidimensional, making them ideal for optimizing robotic joint configurations.

Algorithm Setup and Parameters:

Population and Generations: The algorithm begins with a population of 80 individuals and evolves over 1200 generations. Each individual in the population represents a potential solution, specifically a set of joint configurations for the robotic arm.

Target Position: The target position for the end-effector is determined randomly within the workspace's allowable range, ensuring that the solutions are robust across a variety of possible scenarios:

Algorithm Description:

Initialization: The population is initialized with individuals randomly generated within the joint limits, ensuring a diverse genetic pool for effective exploration of the solution space.

Fitness Function: The fitness function evaluates how well a given joint configuration aligns with the target position, using forward kinematics to compute the end-effector's actual position and then measuring the distance from the target.

$$Fitness = f(DesiredPosition, FK(Chromosome)) \quad (10)$$

Genetic Operations:

Selection: Individuals are selected based on their fitness scores, with higher fitness individuals having a greater chance of passing their genes to the next generation.

Crossover and Mutation: These genetic operators introduce variability and new genetic combinations into the population, crucial for exploring and exploiting the solution space.

Convergence: The algorithm iteratively improves the population's fitness, tracking the best solutions and fitness scores over generations.

The Genetic Algorithm is methodically applied, iterating through the selection of parents, crossover to produce offspring, and mutation to maintain diversity. After running the genetic algorithm, the best joint configuration is identified along with a history of fitness scores, illustrating the progression towards optimal solutions. The convergence of the algorithm and its effectiveness in optimizing the robotic arm's joint angles are evaluated based on these results.

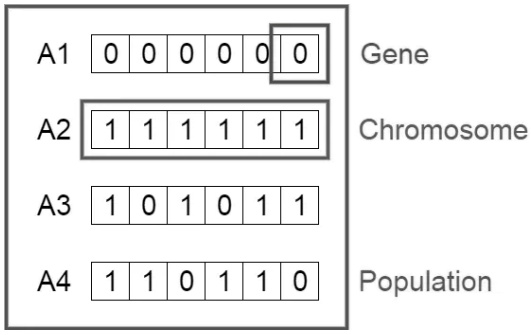


Figure 2.10. : Genetic Algorithm Model[R7].

b) Differential Evolution (DE): Differential Evolution (DE) is a powerful and efficient evolutionary algorithm suited for solving complex optimization problems with non-linear, non-differentiable, and multi-modal objective functions. Its ability to handle high-dimensional spaces and maintain population diversity makes it ideal for robotic applications, particularly for optimizing joint angles to achieve precise end-effector positioning.

Objective Function: The goal of DE in this context is to minimize the Euclidean distance between the robotic arm's end-effector position, calculated through forward kinematics based on the current joint angles, and a desired target position. This objective ensures the end-effector reaches as close as possible to the target, enhancing the robot's accuracy and functionality.

Population Initialization: DE starts with a randomly initialized population within specified bounds, adjusted to respect joint limits, ensuring feasible solutions from the outset.

Core Operations of Differential Evolution:

Mutation: For each member of the population, a mutant vector is generated by adding the weighted difference between two population vectors to a third vector. This mutation step introduces variability and helps explore the solution space.

Crossover: DE employs a crossover mechanism to mix the mutant vector with the existing population member, increasing diversity within the population.

Selection: Each trial vector competes against its corresponding target vector; if the trial achieves a lower objective function value (distance), it replaces the target in the population.

Implementation details are as follows:

Parameters and Execution:

Population Size: 90

Number of Generations: 1000

Mutation Factor and Crossover Probability are set to promote thorough exploration and robust exploitation of the search space.

Tracking and Evaluation: The algorithm tracks the best solution and its score throughout the evolutionary process, printing intermediate scores and responding dynamically to convergence. The implementation concludes with an evaluation of the best solution, assessing how closely the robotic arm's end-effector can reach the target position. The differential evolution process thus ensures that the robotic arm is not only reaching its target accurately but also optimizing the joint movements for optimal performance.

c) Particle Swarm Optimization (PSO): PSO is an evolutionary computation technique that simulates the social behavior observed in animals. It's particularly efficient at navigating complex, high-dimensional search spaces to find optimal solutions, making it an excellent choice for

optimizing robotic arm configurations to achieve precise end-effector positions.

Implementation with PySwarms:

PySwarms Library: The study utilizes PySwarms, a powerful Python library designed for particle swarm optimization. This library facilitates the implementation of PSO, providing tools for handling particle movements and interactions efficiently.

Algorithm Description:

Objective Function: The fitness or objective function evaluates the quality of each solution by calculating the Euclidean distance between the current end-effector position

and a target position. This function is used to guide the swarm towards the minimum distance, indicating the optimal joint configuration.

Core Operations of Particle Swarm Optimization:

Initialization and Parameters: Particles are initialized within given bounds, with each particle representing a potential solution (set of joint angles). The swarm size and the dimensions of the problem are defined based on the robotic arm's degrees of freedom.

Optimization Process: PSO is conducted using the Global Best strategy, where each particle adjusts its velocity and position based on its personal best solution and the global best found by any particle in the swarm. At each iteration, the position of each particle would be updated as

$$X^i(t+1) = X^i(t) + V^i(t+1) \quad (11)$$

while the velocities are updated as well, by the rule

$$V^i(t+1) = wV^i(t) + c_1 r_1 \left(pbest^i - X^i(t)\right) + c_2 r_2 (gbest - X^i(t)) \quad (12)$$

where r_1 and r_2 are random numbers between 0 and 1, constants w, c_1, and c_2 are parameters to the PSO algorithm, and
$pbest^i$ is the position that gives the best f(X) value ever explored by particle i and $gbest$ is that explored by all the particles in the swarm[30].

Evaluation and Visualization Tools: PySwarms provides functionalities to visualize the optimization process and the cost history, helping to analyze the convergence behavior of the swarm.

The optimization process aims to minimize the distance function, with the best set of joint angles returned at the end of the iterations. The final joint configurations and the corresponding end-effector position are then evaluated to ensure accuracy. PSO's application in this context demonstrates its effectiveness in solving the inverse kinematics problem by
finding joint configurations that closely achieve a desired target position. The algorithm's ability to handle complex, nonlinear optimization landscapes ensures that the robotic arm operates efficiently and precisely within its operational constraints.

d) Gradient Descent: Gradient Descent is an iterative optimization algorithm that minimizes a cost function by iteratively moving in the direction of the steepest descent as defined by the negative of the gradient. In the context of robotic arm kinematics, it is used to find the optimal joint angles that minimize the distance between the end-effector's position and a target position, enhancing the robot's accuracy and functionality.

Algorithm Description:

Objective Function: Calculates the Euclidean distance between the current position of the robotic arm's end-effector, determined by forward kinematics, and a specified target position. The aim is to minimize this distance, effectively positioning the end-effector as close to the target as possible.

Core Operations of Gradient Descent:

Initialization: Starts with an initial set of joint angles and iterates over a specified number of iterations to adjust these angles.

Gradient Calculation: Employs a finite difference approach to approximate the gradient of the cost function with respect to each joint angle. This method involves perturbing each joint angle slightly and observing the change in the cost.

Parameter Update: Adjusts the joint angles in the direction that reduces the cost, using a specified learning rate.

Implementation and Application:

Gradient Descent Function: A custom implementation that uses the perturbation method for gradient approximation and updates joint configurations to minimize the distance to a target position.

Tracking: The function records the cost at each iteration to monitor the progression of the optimization, providing insights into the effectiveness of the approach and convergence behavior.

Results and Optimization Impact:

After running the gradient descent, the algorithm outputs the optimized joint angles and a history of costs across iterations. These results showcase the algorithm's ability to refine the robotic arm's configuration, guiding it toward a setup that closely aligns the end-effector with the target position.

e) Simulated Annealing (SA): SA is inspired by the annealing process used in metallurgy. This optimization algorithm is particularly effective for problems with many local minima, like robotic arm kinematics, where it navigates the solution space by simulating the cooling of material to minimize energy states—in this case, the energy being the error in reaching a target position.

Algorithm Description:

Objective Function: The function calculates the Euclidean distance between the current and the desired end-effector positions, derived from the joint angles, aiming to minimize this distance.

Core Operations of Simulated Annealing:

Initialization: Begins with an initial solution generated randomly within the bounds of each joint's range. This solution represents a feasible set of joint angles.

Cooling Schedule: Implements a cooling schedule that decreases the temperature from an initial high value, allowing for extensive exploration initially and more refined exploitation as the temperature lowers.

Neighbor Solution Generation: At each iteration, a new candidate solution is generated by perturbing the current solution slightly within the allowed step size and ensuring that the new joint angles remain within the joint limits.

Acceptance Criterion: A new solution is accepted if it improves the objective function, or with a probability that depends on the difference in cost between the current and new solutions and the current temperature, allowing it to occasionally accept worse solutions to escape local minima.

The algorithm employs an acceptance probability given in Eq.13:

$$P(\Delta E, T) = e^{-\Delta E/T} \quad (13)$$

Where ΔE is the difference in the objective function values between the current and new solutions, and T is the current temperature[31].

Results and Optimization Impact:

After completion, the algorithm provides the optimal joint configurations, the corresponding minimal distance to the target, and histories of costs and temperatures. These results demonstrate the effectiveness of SA in navigating complex solution spaces and refining solutions to precisely control a robotic arm's movement.

E. Evaluation Metrics

1) Mean Absolute Error (MAE):

Definition: The mean absolute error measures the average magnitude of the errors in a set of predictions, without considering their direction. It's the average over the test sample of the absolute differences between prediction and actual observation where all individual differences have equal weight.

Interpretation: A lower MAE indicates better predictive accuracy, with a value of 0 representing perfect prediction. MAE is particularly useful because it is easy to interpret and is not sensitive to outliers.

2) Mean Squared Error (MSE):

Definition: MSE measures the average of the squares of the errors—that is, the average squared difference between the estimated values and the actual value.

Interpretation: MSE is more sensitive to outliers than MAE due to squaring the prediction error. A lower MSE value indicates a better fit to the data.

3) Root Mean Squared Error (RMSE):

Definition: RMSE is the square root of the mean of the squared errors. It assesses how well a regression model predicts or estimates outcomes.

Interpretation: RMSE gives a relatively high weight to large errors, making it useful when large errors are particularly undesirable. Like MSE, a lower RMSE is better.

4) R-squared (R^2):

Definition: R-squared is a statistical measure of how close the data are to the fitted regression line. It is also known as the coefficient of determination.

Interpretation: R^2 represents the proportion of the variance for a dependent variable that's explained by an independent variable or variables in a regression model. A higher R^2 indicates a better fit with values typically between 0 and 1.

5) Explained Variance:

Definition: This metric measures the proportion to which a mathematical model accounts for the variation (dispersion) of a given data set.

Interpretation: Higher values indicate better explanatory power of the model, with a maximum value of 1 indicating perfect explanation.

III. RESULTS

This section presents an evaluation of results obtained by neural networks, regression models, and optimization algorithms, implemented for solving the inverse kinematics of the KUKA iiwa 7 R800 robotic arm. The performance is quantified through several metrics, and cost histories illustrate the performance of optimization processes and the algorithms' efficiency in converging towards a solution.

These metrics not only provide insight into the accuracy and precision of the predictions but also shed light on the models' ability to capture the variance in the data.

A. Neural Networks and Regression Models

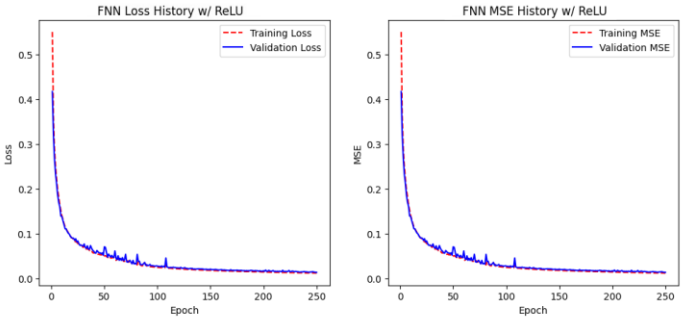
1) *Quantitative Analysis:*

TABLE II
PERFORMANCE METRICS OF ML ALGORITHMS.

Model	MAE	MSE	RMSE	R^2	Exp.Var.
FNN ReLU	0.1621	0.0136	0.1166	0.8833	0.8842
FNN Tanh	0.1629	0.0148	0.1216	0.8794	0.8798
RNN	0.0406	0.0048	<0.001	0.764	0.6705
LSTM	0.1101	0.0181	0.1345	0.2909	0.4327
RFR	0.0045	0.0049	<0.001	0.9991	0.9993
DT	0.0881	0.1315	0.3493	0.9896	0.9687
SVR	0.5140	0.9440	0.9500	0.9250	0.733

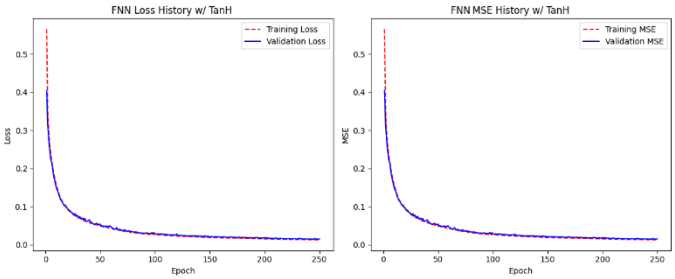
2) *Visualizations:*

a)



(a) (b)

Figure 3.1. FNN Error Percentage with ReLU. (a) MAE, (b) MSE.



(a) (b)

Figure 3.2. FNN Error Percentage with Tanh. (a) MAE, (b) MSE.

The FNN Loss and MSE plots show the model's training and validation loss and MSE during training epochs for both ReLU and TanH activation functions. Both activation functions demonstrate a sharp decline in training loss and MSE, which levels off, indicating that the model quickly learned the training data. The validation loss and MSE curves being close to the training curves suggest that the model generalizes well.

b)

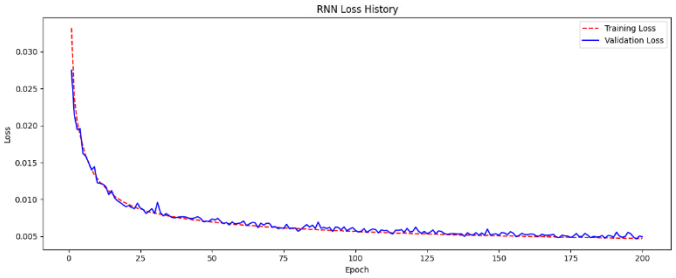


Figure 3.3. RNN Error Percentage.

For the RNN model, the loss plot reveals a steady decline in both training and validation loss, indicating learning stability and good generalization. The validation loss remains close to the training loss throughout, suggesting that the RNN model isn't overfitting.

c)

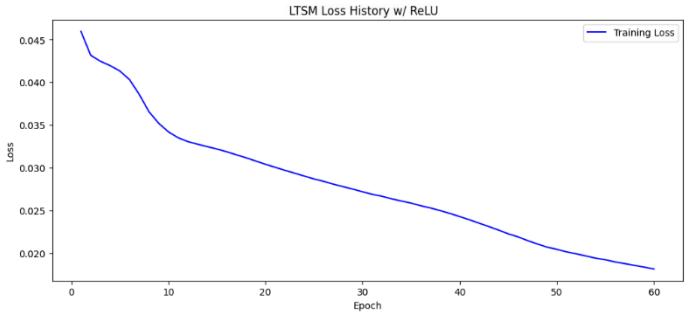


Figure 3.4. LSTM Error Percentage.

The LTSM plot shows a consistent decrease in training loss over epochs, which suggests that the LTSM model was able to learn from the training data effectively. The use of ReLU activation seems to have facilitated the convergence of the loss to a low value.

d)

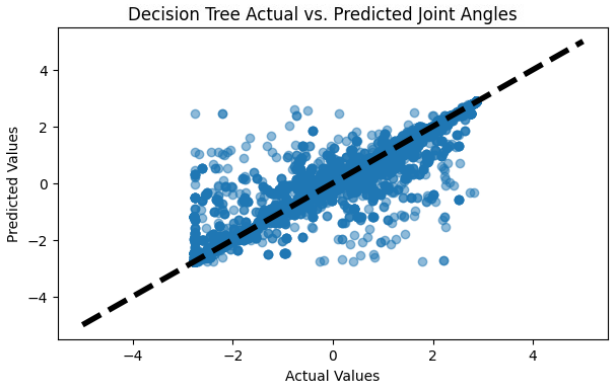


Figure 3.2. Decition Tree Actual vs. Predicted Joint Angles.

The scatter plot for the Decision Tree model shows actual versus predicted joint angles, with the dashed line representing perfect predictions. The concentration of points along this line suggests that the Decision Tree has reasonable predictive accuracy. However, there is noticeable variance, indicating some predictions are off, which is typical for decision tree regressors due to their variance.

e)

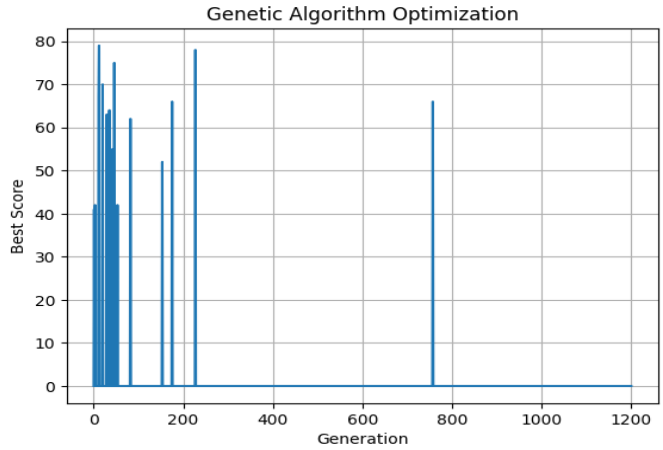


Figure 3.5. Genetic Algorithm Best Score Over Generations.

The Genetic Algorithm (GA) plot displays spikes in the best score, which might indicate that the algorithm sometimes finds significantly better solutions but also has moments of stagnation or potentially worse performance before finding improvements again. The wide variability suggests that GA may be sensitive to the initial population or parameter settings.

f)

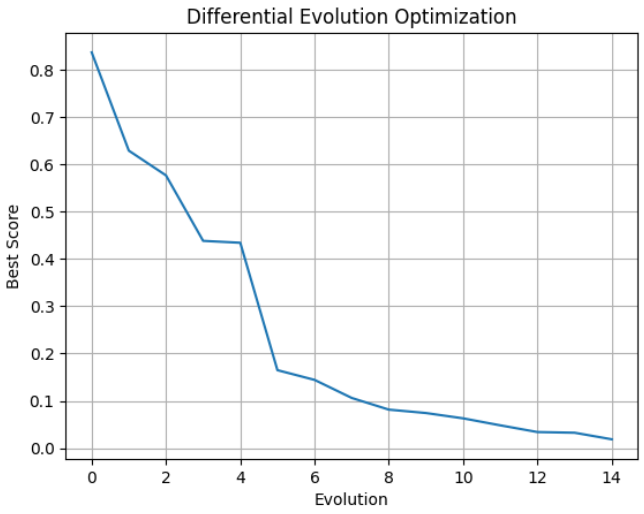


Figure 3.6. Differential Evolution Error Over Better Generations.

The Differential Evolution (DE) plot shows a downward trend in the best score over the evolution process. This indicates the algorithm is improving its solutions over time. The relatively smooth descent suggests that DE avoids getting trapped in local minima, consistently finding better solutions as the iterations progress.

g)

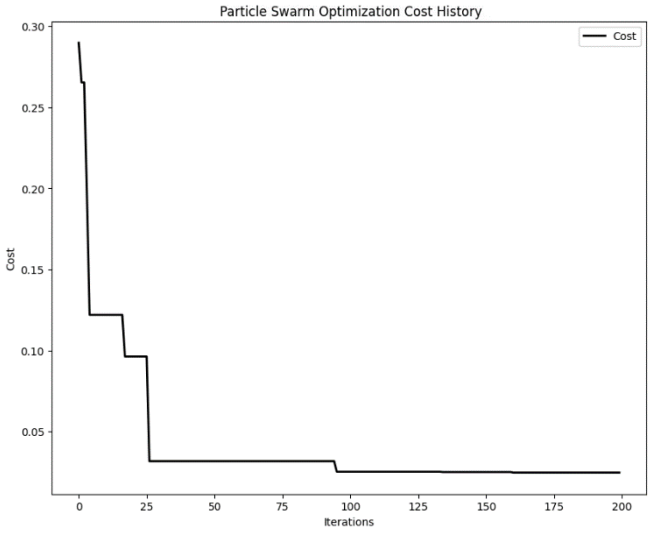


Figure 3.7. Particle Swarm Algorithm Error Over Generations.

The PSO Cost History plot illustrates a rapid initial decrease in cost, indicating fast convergence towards promising areas of the solution space. After this initial phase, improvements slow down, which is common in PSO as particles begin to cluster around the best solution found.

h)

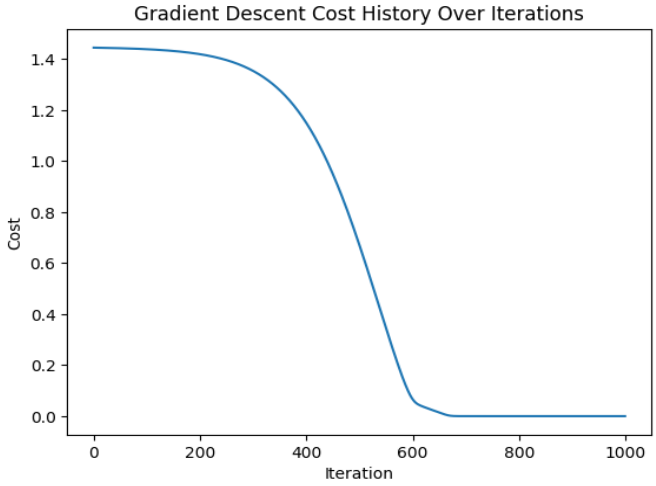
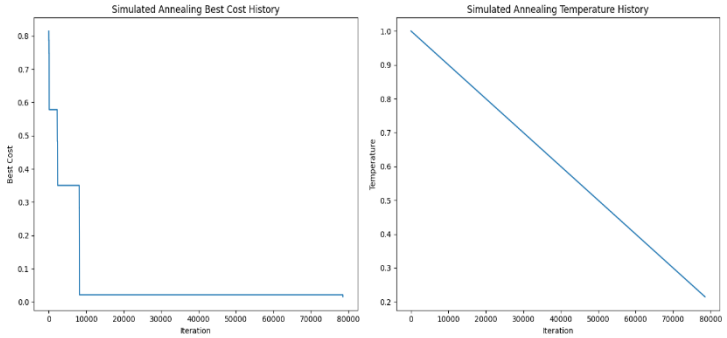


Figure 3.7. Gradient Descent Algorithm Error Over Generations.

The Gradient Descent graph shows a smooth decline in cost, indicating efficient and stable convergence towards an optimal solution. As the iterations continue, the curve becomes more gradual, indicating the algorithm is fine-tuning the parameters and approaching a minimum. The quick initial decrease followed by a plateau suggests an appropriately chosen learning rate and a well-behaved cost function.

j)



(a) (b)

Figure 3.1. Simulated Annealing. (a) Error Percentage, (b) Temperature.

The Simulated Annealing plots show a sharp decrease in best cost at the start, which stabilizes as the temperature decreases. This is characteristic of SA, which allows more exploration at higher temperatures and becomes more exploitative as the temperature drops.

Tables should look like the one below; please keep the format concise:

IV. Conclusions

In this concluding section, we synthesize the insights and discoveries made throughout the study, reflecting on the intersection of machine learning, optimization algorithms, and the dynamic field of robotic inverse kinematics. The investigation has provided a rigorous examination of diverse computational techniques, each contributing uniquely to the overarching goal of enhancing robotic precision and autonomy.

The study's major findings underscore the potential of machine learning to transform the landscape of robotic movement and control. Neural networks, with their deep learning capabilities, have demonstrated notable success in modeling the complex relationships inherent in the kinematics of a 7-DOF robotic arm. Regression models offered a degree of transparency and interpretability in their predictions, while the explored optimization algorithms, each with its philosophical and computational underpinnings, navigated the solution spaces with varying degrees of success and efficiency.

The application of these algorithms has revealed a significant impact on the field of robotics. The study's results contribute to a deeper understanding of the algorithms' behavior in practical scenarios, demonstrating that advanced computational techniques can lead to more nuanced and sophisticated robotic behaviors, ultimately pushing the envelope of what these versatile machines can accomplish.

Looking ahead, the paper identifies several exciting fields for future research:

Complex AI Integration: Incorporating more complex AI models could provide further enhancements in the accuracy and adaptability of robotic systems, allowing them to handle more complex tasks with greater precision.

Real-Time Learning: Exploring real-time learning capabilities would represent a leap forward, enabling robots to adapt on-the-fly to changing environments and tasks, a frontier in autonomous robotics.

Advanced Optimization Techniques: Further investigation into hybrid and advanced optimization techniques could yield more robust and efficient solutions to complex kinematic problems.

Cross-Disciplinary Applications: Applying the learnings from this study to cross-disciplinary problems could unlock new applications and innovations in fields like prosthetics, where robotic precision is critical.

Extensions and Improvements:

The work could be extended in multiple dimensions, such as integrating sensor feedback into learning algorithms to enhance real-world interaction capabilities or scaling the models to accommodate larger, more complex robotic systems. Improvements might also involve optimizing computational efficiency or leveraging unsupervised learning techniques to further refine the models.

This study is a stepping stone, one that not only showcases the potential of sophisticated computational techniques but also lays the groundwork for the next generation of robotics research—a field that remains as limitless as our collective imagination.

Acknowledgments

I would like to extend my deepest gratitude to the Robotics Department of Salford University, whose facilities and stimulating intellectual environment have been instrumental in the realization of this project. Special thanks to my supervisors, Dr. Salem Ameen and Prof Mary He, whose expertise and insights have greatly shaped and guided this work. I am also indebted to Salih Uğur Şah for his constant and multifaceted support during this research. His invaluable assistance has been instrumental in every phase of this work.

I must also thank Mr. Addison Sears-Collins a.k.a. Automatic Addison (https://automaticaddison.com) for his great tutorials and guidance which have been greatly beneficial in helping the author become ROS-literate over the years.

Lastly, I acknowledge the constant support of my peers and family, who have provided both motivation and respite throughout this demanding academic endeavor.

REFERENCES

[1] Al-Khafaji, H. and Jweeg, M. (2017). Solving the inverse kinematic equations of elastic robot arm utilizing neural network. Al-Khwarizmi Engineering Journal, 13(1), 13-25. https://doi.org/10.22153/kej.2017.11.002

[2] Trutman, P. et al. (2020) Globally optimal solution to inverse kinematics of 7DOF serial manipulator, arXiv.org. Available at: https://arxiv.org/abs/2007.12550.

[3] Wang, X., Cao, J., Liu, X., Chen, L., & Hu, H. (2020). An Enhanced Step-Size Gaussian Damped Least Squares Method Based on Machine Learning for Inverse Kinematics of Redundant Robots. IEEE Access, 8, 68057-68067.

[4] . H. -C. Huang, C. -P. Chen and P. -R. Wang, "Particle swarm optimization for solving the inverse kinematics of 7-DOF robotic manipulators," 2012 IEEE International Conference on Systems, Man, and Cybernetics (SMC), Seoul, Korea (South), 2012, pp. 3105-3110, doi: 10.1109/ICSMC.2012.6378268.

[5] Z. Bingul, H. M. Ertunc and C. Oysu, "Comparison of inverse kinematics solutions using neural network for 6R robot manipulator with offset," 2005 ICSC Congress on Computational Intelligence Methods and Applications, Istanbul, Turkey, 2005, pp. 5 pp.-, doi: 10.1109/CIMA.2005.1662342.

[6] Phuoc, L.M., Martinet, P., Lee, S. et al. Damped least square based genetic algorithm with Ggaussian distribution of damping factor for singularity-robust inverse kinematics. J Mech Sci Technol 22, 1330–1338 (2008). https://doi.org/10.1007/s12206-008-0427-4

[7] Denavit, J. and Hartenberg, R. (1955). A kinematic notation for lower-pair mechanisms based on matrices. Journal of Applied Mechanics, 22(2), 215-221. https://doi.org/10.1115/1.4011045

[8] Spong, M.W. and Vidyasagar, M. (1991) Robot Dynamics and control, Wiley.com. Available at: https://www.wiley.com/en-us/Robot+Dynamics+and+Control-p-9780471612438 (Accessed: 16 April 2024).

[9] Ricardo Xavier da Silva, S., Schnitman, L. and Cesca Filho, V. (2021) 'A solution of the inverse kinematics problem for a 7-degrees-of-freedom serial redundant manipulator using Gröbner Bases theory', Mathematical Problems in Engineering, 2021, pp. 1–14. doi:10.1155/2021/6680687.

[10] https://www.kuka.com/-/media/kuka-downloads/imported/8350ff3ca11642998dbdc81dcc2ed44c/0000246832_en.pdf?rev=d4eeb820921d4e2199eb9e20902622bd&hash=A24188E9EFFC1DD9E1BCE848AC696C21

[11] https://ubuntu.com/robotics/what-is-ros

[12] https://roboticsknowledgebase.com/wiki/tools/gazebo-simulation

[13] Hennersperger C, Fuerst B, Virga S, Zettinig O, Frisch B, Neff T, Navab N. Towards MRI-Based Autonomous Robotic US Acquisitions: A First Feasibility Study. IEEE Trans Med Imaging. 2017 Feb;36(2):538-548. doi: 10.1109/TMI.2016.2620723. Epub 2016 Oct 24. PMID: 27831861.

[14] Robot arm model description taken from: https://github.com/IFL-CAMP/iiwa_stack

[15] Vision Inspection with Neural Networks - ResearchGate. Available at: https://www.researchgate.net/publication/329586439_Vision_Inspection_with_Neural_Networks.

[16] Chen, Z. and Cao, F. (2020). Construction of feedforward neural networks with simple architectures and approximation abilities. Mathematical Methods in the Applied Sciences, 44(2), 1788-1795. https://doi.org/10.1002/mma.6876

[17] https://builtin.com/machine-learning/relu-activation-function

[18] CSC 411 lecture 10: Neural networks I. Available at: https://www.cs.toronto.edu/~jlucas/teaching/csc411/lectures/lec10_handout.pdf (Accessed: 17 April 2024).

[19] Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. "Deep Learning." MIT Press, 2016.

[20] https://towardsdatascience.com/implementation-of-rnn-lstm-and-gru-a4250bf6c090

[21] Gao, M., Shi, G., & Li, S. (2018). Online prediction of ship behavior with automatic identification system sensor data using bidirectional long short-term memory recurrent neural network. Sensors, 18(12), 4211. https://doi.org/10.3390/s18124211

[22] https://d2l.ai/chapter_recurrent-modern/lstm.html

[23] https://neptune.ai/blog/random-forest-regression-when-does-it-fail-and-why

[24] Chen, X., Zheng, H., Wang, H., & Yan, T. (2022). Can machine learning algorithms perform better than multiple linear regression in predicting nitrogen excretion from lactating dairy cows. Scientific Reports, 12(1). https://doi.org/10.1038/s41598-022-16490-y

[25] https://python-course.eu/machine-learning/decision-trees-in-python.php

[26] https://statisticsbyjim.com/regression/mean-squared-error-mse/

[27] https://uk.mathworks.com/discovery/support-vector-machine.html

[28] Cortes, C., Vapnik, V. Support-vector networks. Mach Learn 20, 273–297 (1995). https://doi.org/10.1007/BF00994018

[29] https://towardsdatascience.com/introduction-to-genetic-algorithms-including-example-code-e396e98d8bf3

[30] https://machinelearningmastery.com/a-gentle-introduction-to-particle-swarm-optimization/

[31] J. Chem. Phys. 21, 1087 (1953); doi: 10.1063/1.1699114

APPENDIX I

A. *Google Drive Link To Project Files*

https://drive.google.com/drive/folders/1OVNqOPZk-SfVAmAaesiQH3liYoqACNr0

B. *Raw Joint Information Data*

Figure I. Raw Joint Information Data.

	Time	header.seq	header.stamp.secs	header.stamp.nsecs	header.frame_id	name	position_0	position_1	position_2	position_3
0	1054.683	105380	1054	681000000	NaN	['iiwa_joint_1', 'iiwa_joint_2', 'iiwa_joint_3...	0.000013	0.000071	-0.000026	-0.000045
1	1054.691	105381	1054	691000000	NaN	['iiwa_joint_1', 'iiwa_joint_2', 'iiwa_joint_3...	0.000013	0.000071	-0.000026	-0.000045
2	1054.701	105382	1054	701000000	NaN	['iiwa_joint_1', 'iiwa_joint_2', 'iiwa_joint_3...	0.000013	0.000071	-0.000026	-0.000045
3	1054.711	105383	1054	711000000	NaN	['iiwa_joint_1', 'iiwa_joint_2', 'iiwa_joint_3...	0.000013	0.000071	-0.000026	-0.000045
4	1054.721	105384	1054	721000000	NaN	['iiwa_joint_1', 'iiwa_joint_2', 'iiwa_joint_3...	0.000013	0.000071	-0.000026	-0.000045
...	...	...	...	...	...	...	...	...	...	...
29996	1354.639	135376	1354	641000000	NaN	['iiwa_joint_1', 'iiwa_joint_2', 'iiwa_joint_3...	2.351751	-0.147447	-1.175521	1.350805
29997	1354.651	135377	1354	651000000	NaN	['iiwa_joint_1', 'iiwa_joint_2', 'iiwa_joint_3...	2.348598	-0.148008	-1.172338	1.348688
29998	1354.661	135378	1354	661000000	NaN	['iiwa_joint_1', 'iiwa_joint_2', 'iiwa_joint_3...	2.345445	-0.148570	-1.169154	1.346571
29999	1354.671	135379	1354	671000000	NaN	['iiwa_joint_1', 'iiwa_joint_2', 'iiwa_joint_3...	2.342291	-0.149132	-1.165970	1.344454
30000	1354.681	135380	1354	681000000	NaN	['iiwa_joint_1', 'iiwa_joint_2', 'iiwa_joint_3...	2.339138	-0.149693	-1.162786	1.342337

C. Raw End Effector Position Information Data

	Time	name	pose	twist
0	1054.683	['ground_plane::link', 'iiwa::iiwa_link_0', 'i...	[position: \n x: 0.0\n y: 0.0\n z: 0.0\nori...	[linear: \n x: 0.0\n y: 0.0\n z: 0.0\nangul...
1	1054.683	['ground_plane::link', 'iiwa::iiwa_link_0', 'i...	[position: \n x: 0.0\n y: 0.0\n z: 0.0\nori...	[linear: \n x: 0.0\n y: 0.0\n z: 0.0\nangul...
2	1054.683	['ground_plane::link', 'iiwa::iiwa_link_0', 'i...	[position: \n x: 0.0\n y: 0.0\n z: 0.0\nori...	[linear: \n x: 0.0\n y: 0.0\n z: 0.0\nangul...
3	1054.683	['ground_plane::link', 'iiwa::iiwa_link_0', 'i...	[position: \n x: 0.0\n y: 0.0\n z: 0.0\nori...	[linear: \n x: 0.0\n y: 0.0\n z: 0.0\nangul...
4	1054.684	['ground_plane::link', 'iiwa::iiwa_link_0', 'i...	[position: \n x: 0.0\n y: 0.0\n z: 0.0\nori...	[linear: \n x: 0.0\n y: 0.0\n z: 0.0\nangul...
...	...	...	...	...
299989	1354.681	['ground_plane::link', 'iiwa::iiwa_link_0', 'i...	[position: \n x: 0.0\n y: 0.0\n z: 0.0\nori...	[linear: \n x: 0.0\n y: 0.0\n z: 0.0\nangul...
299990	1354.681	['ground_plane::link', 'iiwa::iiwa_link_0', 'i...	[position: \n x: 0.0\n y: 0.0\n z: 0.0\nori...	[linear: \n x: 0.0\n y: 0.0\n z: 0.0\nangul...
299991	1354.681	['ground_plane::link', 'iiwa::iiwa_link_0', 'i...	[position: \n x: 0.0\n y: 0.0\n z: 0.0\nori...	[linear: \n x: 0.0\n y: 0.0\n z: 0.0\nangul...
299992	1354.681	['ground_plane::link', 'iiwa::iiwa_link_0', 'i...	[position: \n x: 0.0\n y: 0.0\n z: 0.0\nori...	[linear: \n x: 0.0\n y: 0.0\n z: 0.0\nangul...
299993	1354.681	['ground_plane::link', 'iiwa::iiwa_link_0', 'i...	[position: \n x: 0.0\n y: 0.0\n z: 0.0\nori...	[linear: \n x: 0.0\n y: 0.0\n z: 0.0\nangul...

Figure II. Raw End Effector Position Information Data.

D. Semi-Processed End Effector Position Information Data

[position: \n x: 0.0\n y: 0.0\n z: 0.0\norientation: \n x: 0.0\n y: 0.0\n z: 0.0\n w: 1.0, position: \n x: 5.528009865141126e-08\n y: -3.259223954579142e-08\n z: 3.8642248596787443e-07\norientation: \n x: 1.27621350506052e-08\n y: -9.166620881576895e-07\n z: -1.1954558673465723e-07\n w: 0.9999999999995728, position: \n x: 1.6411417776039058e-06\n y: 2.7126937853644284e-07\n z: 0.15000009398729025\norientation: \n x: -2.6008785588195955e-07\n y: -7.943889761683671e-07\n z: 6.366197425863224e-06\n w: 0.9999999999793864, position: \n x: 1.7826460398504763e-06\n y: 1.9706185802602994e-07\n z: 0.33999991190534 75\norientation: \n x: 2.0373598748012738e-05\n y: 0.7071081939477214\n z: 0.707105367594735\n w: -2.7488590820232617e-05, position: \n x: 1.6685314128763346e-05\n y: 7.605423252010559e-07\n z: 0.5499997691965299\norientation: \n x: -1.7241159171749294e-06\n y: -3.4727418375931076e-05\n z: 7.70399764780472e-06\n w: -0.999999999365841, position: \n x: 2.982939556505049e-05\n y: -1.1026135089835955e-07\n z: 0.7399994053384645\norientation: \n x: -0.7071102365180965\n y: -3.487087125846861e-05\n z: 4.595618207130001e-05\n w: -0.7071033234848876, position: \n x: 5.361324939952988e-05\n y: -2.617100734835899e-06\n z: 0.9499991531355039\norientation: \n x: -5.7857692233718445e-05\n y: 3.7694976671240333e-06\n z: -0.9999999974089023\n w: -4.266701008226583e-05, position: \n x: 7.032380289830775e-05\n y: -0.06070425841409702\n z: 1.1399983341836095\norientation: \n x: -7.118787130367233e-05\n y: -0.7071041851772556\n z: -0.7071093735170815\n w: 1.1017128010419203e-05, position: \n x: 8.483071478015996e-05\n y: -5.035803146175022e-06\n z: 1.2209985494984597\norientation: \n x: 4.998606839501499e-06\n y: 5.634515291003493e-05\n z: -6.150433267404365e-05\n w: 0.9999999965087275]'

Figure III. Semi-Processed End Effector Position Information Data.

E. Fully-Processed Joint Angles and End Effector Position Information Data.

	Time	x	y	z	theta0	theta1	theta2	theta3	theta4	theta5	theta6
0	1054.683	8.483071478015996e-05	-5.035803146175022e-06	1.2209985494984597	0.000013	0.000071	-0.000026	-0.000045	-0.000070	0.000004	-0.000038
1	1054.683	8.429867892710055e-05	-5.042569954274989e-06	1.2209985673775714	0.000013	0.000071	-0.000026	-0.000045	-0.000070	0.000004	-0.000038
2	1054.683	8.483071477987694e-05	-5.035803146175022e-06	1.2209985494984597	0.000013	0.000071	-0.000026	-0.000045	-0.000070	0.000004	-0.000038
3	1054.683	8.4298678927 6e-05	-5.042569954274989e-06	1.2209985673775714	0.000013	0.000071	-0.000026	-0.000045	-0.000070	0.000004	-0.000038
4	1054.691	8.429867892703463e-05	-5.042569954274989e-06	1.2209985673775714	0.000013	0.000071	-0.000026	-0.000045	-0.000070	0.000004	-0.000038
...	...	...	...	...	...	...	...	...	...	...	...
30289	1354.681	-0.15420584950229	-0.4714431640968517	0.8014031161208446	2.339138	-0.149693	-1.162786	1.342337	-1.004323	-0.400514	-0.414176
30290	1354.681	-0.15415708849764148	-0.4714957852221 4886	0.8014517067417991	2.339138	-0.149693	-1.162786	1.342337	-1.004323	-0.400514	-0.414176
30291	1354.681	-0.15410853930996982	-0.4715480583206 3773	0.8015005564640846	2.339138	-0.149693	-1.162786	1.342337	-1.004323	-0.400514	-0.414176
30292	1354.681	-0.15405977015618 51	-0.4716006537270218	0.8015492339847656	2.339138	-0.149693	-1.162786	1.342337	-1.004323	-0.400514	-0.414176
30293	1354.681	-0.15401120846115976	-0.4716529004799701	0.801598171086627	2.339138	-0.149693	-1.162786	1.342337	-1.004323	-0.400514	-0.414176

Figure IV. Fully-Processed Joint Angles and End Effector Position Information Data.